

Task 3: Data Logger :Sensor → SD Card

1. Project Overview

Data logging is the process of collecting, recording, and storing data over a period of time for monitoring and analysis. Data loggers are widely used in industries, environmental monitoring, agriculture, healthcare, weather stations, and research laboratories. This project demonstrates a simple data logging system using an Arduino Uno. The Arduino continuously reads temperature data from a sensor at regular intervals and records the readings. In this project, the Serial Monitor is used to simulate the data logging process, where timestamped temperature values are displayed continuously. The project introduces the concepts of sensor data acquisition, periodic data recording, and real-time monitoring using embedded systems.

2. Aim / Objective

The objective of this project is to develop a simple data logging system using an Arduino Uno that continuously reads temperature data and records the readings at regular time intervals. The collected data is displayed on the Serial Monitor, simulating the functionality of a data logger for monitoring and analysis.

3. Components Required

- Arduino Uno
- TMP36 Temperature Sensor (*or simulated temperature values*)
- Breadboard
- Jumper Wires

4. Software Required

- Arduino IDE
- Tinkercad Circuits Simulator (*optional*)
- Serial Monitor

5. Theory

A data logger is an electronic system designed to automatically collect and store information from sensors over time. The stored data can later be analyzed to understand environmental conditions, detect abnormalities, or evaluate system performance. Data logging plays an important role in scientific research, industrial automation, agriculture, weather forecasting, and quality control.

In this project, the Arduino Uno acts as the central processing unit that continuously reads temperature values from a temperature sensor. The analog voltage produced by the sensor is converted into digital values using the Arduino's built-in Analog-to-Digital Converter (ADC). These values are then converted into temperature readings in degrees Celsius.

Instead of storing the readings on an SD card, the collected data is displayed on the Arduino Serial Monitor along with a timestamp generated using the Arduino's internal timer (`millis()`). This approach simulates the operation of a real data logger and allows continuous observation of sensor readings without requiring additional storage hardware.

The project demonstrates the fundamental concepts of periodic data acquisition, sensor interfacing, timing functions, and serial communication, which are essential building blocks for more advanced data acquisition and IoT-based monitoring systems.

6. Working Principle

The Arduino continuously reads the temperature from the sensor at fixed time intervals. Each reading is processed and converted into a temperature value in degrees Celsius. The Arduino records the elapsed time using its internal timer and displays both the timestamp and temperature reading on the Serial Monitor. This process repeats continuously, thereby simulating the operation of a real-time data logging system.

7. Algorithm

1. Start the program.
2. Initialize the Serial Monitor.
3. Read the temperature value from the sensor.
4. Convert the analog value into temperature.
5. Read the current elapsed time using the Arduino timer.
6. Display the timestamp and temperature on the Serial Monitor.
7. Wait for the specified logging interval.
8. Repeat the process continuously.

8. How to Run

1. Open the Arduino IDE or Tinkercad simulation.
2. Verify the program and upload it to the Arduino (or start the simulation).
3. Open the Serial Monitor.
4. Observe the timestamped temperature readings displayed at regular intervals.
5. Continue monitoring the data as it is logged continuously.

Source Code:

```
#include <SPI.h>
#include <SD.h>
#include <Wire.h>
#include <RTCLib.h>
#include <DHT.h>
```

```
#define DHTPIN 4
#define DHTTYPE DHT11
#define LDR_PIN A0
#define SD_CS 10

DHT dht(DHTPIN, DHTTYPE);
RTC_DS3231 rtc;
File dataFile;

void setup() {
  Serial.begin(9600);
  delay(2000);

  dht.begin();
  Wire.begin();

  if (!rtc.begin()) {
    Serial.println("RTC not found");
    while (1);
  }

  if (!SD.begin(SD_CS)) {
    Serial.println("SD card failed");
    while (1);
  }

  if (!SD.exists("data.csv")) {
```

```
dataFile = SD.open("data.csv", FILE_WRITE);  
dataFile.println("Date,Time,Temperature,Humidity,Light");  
dataFile.close();  
}
```

```
Serial.println("System Ready");  
}
```

```
void loop() {  
  logDataToSD();  
  readFileAndPrint();  
  delay(5000); // sampling rate  
}
```

```
void logDataToSD() {  
  DateTime now = rtc.now();  
  
  float temp = dht.readTemperature();  
  float hum = dht.readHumidity();  
  int lightVal = analogRead(LDR_PIN);  
  
  if (isnan(temp) || isnan(hum)) {  
    Serial.println("DHT error");  
    return;  
  }
```

```
dataFile = SD.open("data.csv", FILE_WRITE);  
if (dataFile) {
```

```
dataFile.print(now.year()); dataFile.print("-");  
dataFile.print(now.month()); dataFile.print("-");  
dataFile.print(now.day()); dataFile.print(",");
```

```
dataFile.print(now.hour()); dataFile.print(":");  
dataFile.print(now.minute()); dataFile.print(":");  
dataFile.print(now.second()); dataFile.print(",");
```

```
dataFile.print(temp); dataFile.print(",");  
dataFile.print(hum); dataFile.print(",");  
dataFile.println(lightVal);
```

```
dataFile.close();  
Serial.println("Data logged to SD");
```

```
}  
}
```

```
void readFileAndPrint() {  
    Serial.println("---- CSV FILE CONTENT ----");
```

```
    dataFile = SD.open("data.csv");
```

```
    if (dataFile) {
```

```
        while (dataFile.available()) {
```

```
            Serial.write(dataFile.read());
```

```
        }
```

```
        dataFile.close();
```

```
    } else {
```

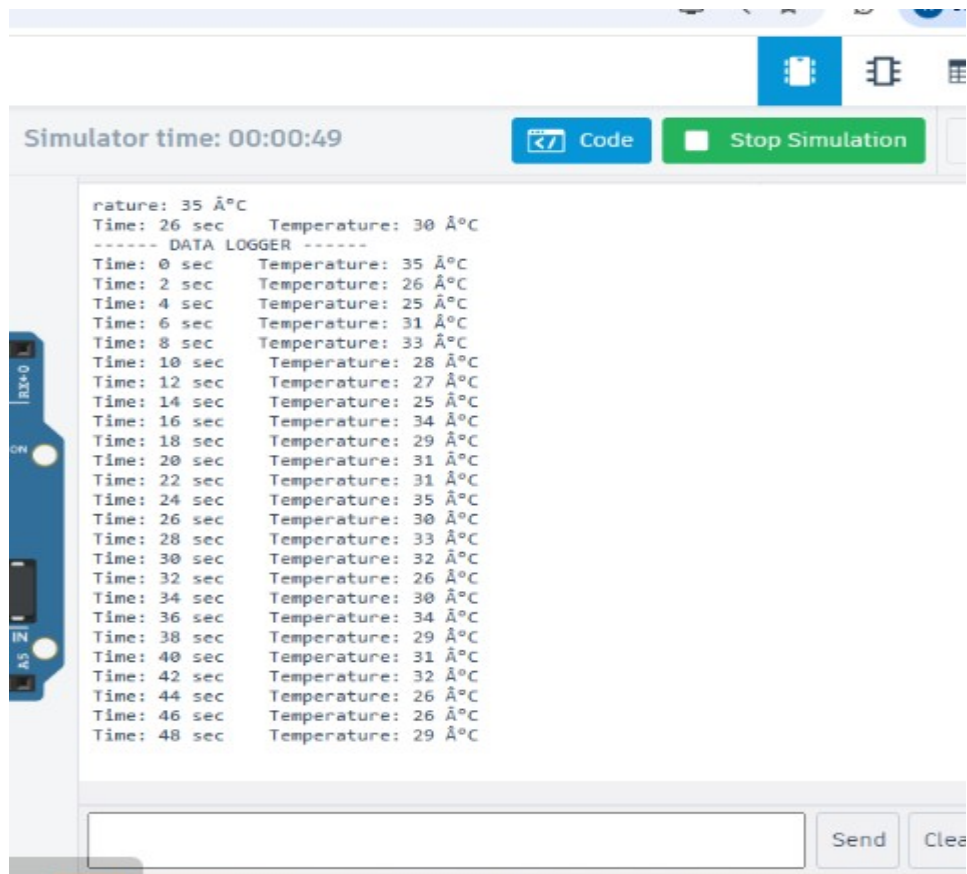
```
        Serial.println("File open failed");
```

```
}
```

```
Serial.println("---- END OF FILE ----");
```

```
}
```

Output:



9.Applications

- Environmental monitoring
- Weather stations
- Greenhouse monitoring
- Industrial process monitoring
- Agricultural monitoring systems
- Laboratory experiments
- Medical data collection
- IoT-based monitoring systems

10. Advantages

- Enables continuous monitoring of sensor data.
- Simple and low-cost implementation.
- Easy to modify for different types of sensors.

- Supports periodic and automated data collection.
- Useful for data analysis and system monitoring.
- Can be expanded to support cloud storage or SD card logging.

11. Skills Gained

- Arduino programming using C/C++.
- Temperature sensor interfacing.
- Serial communication using the Serial Monitor.
- Real-time data acquisition.
- Data logging concepts.
- Embedded systems programming.
- Simulation and debugging using Arduino IDE.

12. Future Scope

This project can be enhanced by integrating an SD card module to permanently store the recorded data. The logged data can also be uploaded to cloud platforms using Wi-Fi-enabled microcontrollers such as the ESP32 for remote monitoring. Additional sensors such as humidity, pressure, soil moisture, or gas sensors can be incorporated to create a comprehensive environmental monitoring system. The project can further be integrated with graphical dashboards for real-time visualization and analysis of historical data.

13. Conclusion

The Data Logger Using Arduino project successfully demonstrates the process of collecting and recording temperature data at regular intervals using an Arduino Uno. By displaying timestamped sensor readings on the Serial Monitor, the project effectively simulates the operation of a real-time data logging system. It provides practical knowledge of sensor interfacing, serial communication, timing functions, and data acquisition techniques. This project serves as a strong foundation for developing advanced monitoring systems that incorporate permanent data storage, wireless communication, and cloud-based analytics.

Internship :

Completed as part of Embedded Systems Internship – Intern spark